

SHETH L.U.J. AND SIR M.V. COLLEGE
PRACTICAL NO.: -2
SUBJECT: -Cyber & Information Security

Aim: Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

INPUT:-

```
import math

def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def mod_inverse(e, phi):
    for d in range(2, phi):
        if (d * e) % phi == 1:
            return d
    return None

def generate_keys():
    p = int(input("Enter prime number p: "))
    q = int(input("Enter prime number q: "))

    n = p * q
    phi = (p - 1) * (q - 1)

    e = 2
    while e < phi:
        if gcd(e, phi) == 1:
            break
        e += 1

    d = mod_inverse(e, phi)

    return ((e, n), (d, n))

def encrypt(public_key, plaintext):
    e, n = public_key
    cipher = [pow(ord(char), e, n) for char in plaintext]
```

SHETH L.U.J. AND SIR M.V. COLLEGE
PRACTICAL NO.:2
SUBJECT:-Cyber & Information Security

```
        return cipher

def decrypt(private_key, ciphertext):
    d, n = private_key
    plain = ''.join([chr(pow(char, d, n)) for char in ciphertext])
    return plain

public_key, private_key = generate_keys()

print("\nPublic Key:", public_key)
print("Private Key:", private_key)

message = input("\nEnter Message: ")

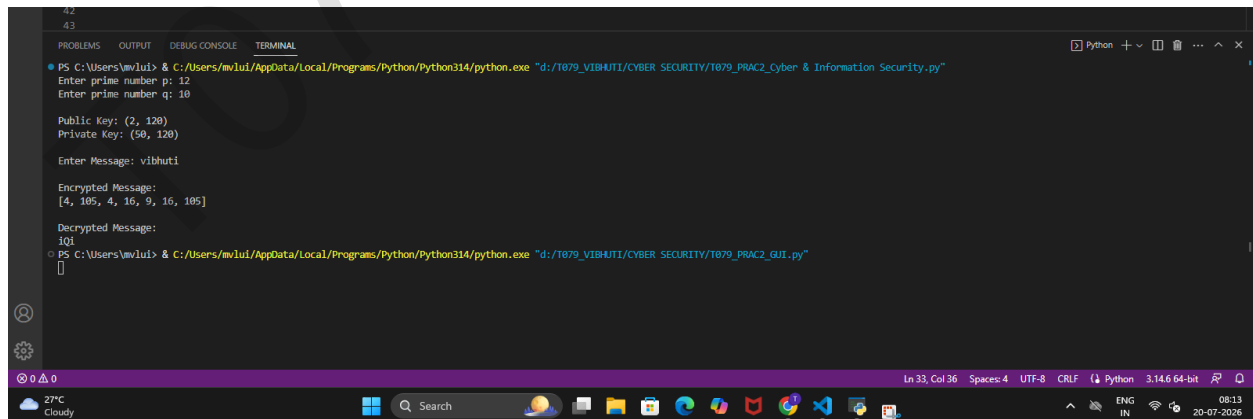
cipher = encrypt(public_key, message)

print("\nEncrypted Message:")
print(cipher)

plain = decrypt(private_key, cipher)

print("\nDecrypted Message:")
print(plain)
```

OUTPUT:-



```
PS C:\Users\m\lul> & C:\Users\m\lul\AppData\Local\Programs\Python\Python314\python.exe "d:/T079_VIBHUTI/CYBER SECURITY/T079_Prac2_Cyber & Information Security.py"
Enter prime number p: 12
Enter prime number q: 10

Public Key: (2, 120)
Private Key: (50, 120)

Enter Message: vibhuti

Encrypted Message:
[4, 105, 4, 16, 9, 16, 105]

Decrypted Message:
vibhuti
PS C:\Users\m\lul> & C:\Users\m\lul\AppData\Local\Programs\Python\Python314\python.exe "d:/T079_VIBHUTI/CYBER SECURITY/T079_Prac2_GUI.py"
```

SHETH L.U.J. AND SIR M.V. COLLEGE
PRACTICAL NO.:2
SUBJECT:-Cyber & Information Security

GUI

INPUT:-

```
import tkinter as tk
from tkinter import messagebox

def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def mod_inverse(e, phi):
    for d in range(2, phi):
        if (d * e) % phi == 1:
            return d
    return None

def generate_keys():
    try:
        p = int(entry_p.get())
        q = int(entry_q.get())

        global public_key, private_key

        n = p * q
        phi = (p - 1) * (q - 1)

        e = 2
        while e < phi:
            if gcd(e, phi) == 1:
                break
            e += 1

        d = mod_inverse(e, phi)

        public_key = (e, n)
        private_key = (d, n)
```

SHETH L.U.J. AND SIR M.V. COLLEGE
PRACTICAL NO.:2
SUBJECT:-Cyber & Information Security

```
lbl_public.config(text="Public Key : " + str(public_key))
lbl_private.config(text="Private Key : " + str(private_key))

except:
    messagebox.showerror("Error", "Enter Valid Prime Numbers")

def encrypt():
    message = entry_message.get()

    e, n = public_key

    cipher = [pow(ord(ch), e, n) for ch in message]

    entry_cipher.delete(0, tk.END)
    entry_cipher.insert(0, str(cipher))

def decrypt():
    try:
        cipher = eval(entry_cipher.get())

        d, n = private_key

        plain = ''.join(chr(pow(ch, d, n)) for ch in cipher)

        entry_plain.delete(0, tk.END)
        entry_plain.insert(0, plain)

    except:
        messagebox.showerror("Error", "Invalid Cipher")

root = tk.Tk()
root.title("RSA Encryption & Decryption")
root.geometry("550x450")

public_key = ()
```

SHETH L.U.J. AND SIR M.V. COLLEGE
PRACTICAL NO.:2
SUBJECT:-Cyber & Information Security

```
private_key = ()

tk.Label(root, text="Prime Number p").pack()
entry_p = tk.Entry(root, width=30)
entry_p.pack()

tk.Label(root, text="Prime Number q").pack()
entry_q = tk.Entry(root, width=30)
entry_q.pack()

tk.Button(root, text="Generate Keys", command=generate_keys).pack(pady=10)

lbl_public = tk.Label(root, text="Public Key")
lbl_public.pack()

lbl_private = tk.Label(root, text="Private Key")
lbl_private.pack()

tk.Label(root, text="Enter Message").pack()
entry_message = tk.Entry(root, width=40)
entry_message.pack()

tk.Button(root, text="Encrypt", command=encrypt).pack(pady=5)

tk.Label(root, text="Cipher Text").pack()
entry_cipher = tk.Entry(root, width=50)
entry_cipher.pack()

tk.Button(root, text="Decrypt", command=decrypt).pack(pady=5)

tk.Label(root, text="Decrypted Message").pack()
entry_plain = tk.Entry(root, width=40)
entry_plain.pack()

root.mainloop()
```

SHETH L.U.J. AND SIR M.V. COLLEGE
PRACTICAL NO.:-2
SUBJECT:-Cyber & Information Security

OUTPUT:-

The screenshot displays a Python GUI application titled "RSA Encryption & Decryption". The interface includes the following elements:

- Prime Number p:** A text input field containing the value "20".
- Prime Number q:** A text input field containing the value "40".
- Generate Keys:** A button that, when clicked, generates the RSA keys.
- Public Key:** A label displaying the generated public key as $(2, 800)$.
- Private Key:** A label displaying the generated private key as $(371, 800)$.
- Enter Message:** A text input field containing the message "vibhuti".
- Encrypt:** A button that, when clicked, encrypts the message.
- Cipher Text:** A text input field displaying the encrypted message as the list $[324, 625, 4, 416, 89, 656, 625]$.
- Decrypt:** A button that, when clicked, decrypts the cipher text.
- Decrypted Message:** A text input field displaying the decrypted message as "vibhuti".

At the bottom of the window, there is a status bar that reads "Public Key: (2, 120)".